CHƯƠNG 3 · BÀI 2

3.2. Kế thừa (Inheritance)

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Hiểu tính kế thừa và lợi ích của nó
- Viết lớp con kế thừa lớp cha, gọi phương thức lớp cha bằng `super()`
- Ghi đè (override) phương thức của lớp cha

1. Kế thừa là gì

Kế thừa (inheritance) cho phép lớp con dùng lại thuộc tính và phương thức của lớp cha, giúp tránh lặp code. Lớp con được khai báo bằng cách đặt tên lớp cha trong dấu ngoặc.

```
class Nguoi:
    def __init__(self, ten):
        self.ten = ten
    def gioi_thieu(self):
        print("Tôi là", self.ten)

class SinhVien(Nguoi):
    def __init__(self, ten, mssv):
        super().__init__(ten)
        self.mssv = mssv
```

2. Hàm super()

Hàm `super()` trả về đối tượng đại diện cho lớp cha, dùng để gọi phương thức của lớp cha, thường là `__init__`, để lớp con không phải viết lại phần khởi tạo chung.

3. Ghi đè phương thức

Lớp con có thể định nghĩa lại một phương thức của lớp cha với cùng tên để thay đổi hành vi. Đây là cơ sở của đa hình: cùng một lời gọi phương thức nhưng mỗi đối tượng xử lý theo cách riêng.

```
class SinhVien(Nguoi):
    def gioi_thieu(self):
```

```
        super().gioi_thieu()
        print("MSSV:", self.mssv)
```

4. Kiểm tra quan hệ kế thừa

`isinstance(sv, Nguoi)` cho biết đối tượng có thuộc lớp (hoặc lớp cha của lớp) đó hay không; `issubclass(SinhVien, Nguoi)` kiểm tra quan hệ giữa hai lớp. Python hỗ trợ đa kế thừa, tức một lớp có thể kế thừa nhiều lớp cha.

Tóm tắt

- Lớp con kế thừa thuộc tính, phương thức của lớp cha.
- `super()` gọi lớp cha; ghi đè phương thức để thay đổi hành vi.
- `isinstance` và `issubclass` kiểm tra quan hệ kế thừa.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết lớp Dong_vat và hai lớp con Cho, Meo cùng ghi đè phương thức `keu()`.
2. Viết lớp NhanVien kế thừa Nguoi, bổ sung thuộc tính lương và phương thức tính lương.
3. Giải thích sự khác nhau giữa ghi đè và gọi `super()` trong một phương thức.

Kế thừa (Inheritance) — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: học viên thường và học viên học bổng có chung thông tin nhưng cách tính học phí khác. Kế thừa phù hợp quan hệ "là một"; thành phần được chứa bên trong nên cân nhắc composition.

```
class Learner:
    def __init__(self, name):
        self.name = name
    def tuition(self):
        return 100

class ScholarshipLearner(Learner):
    def tuition(self):
        return super().tuition() * 0.5

assert ScholarshipLearner("An").tuition() == 50
assert ScholarshipLearner("An").name == "An"
```

Tự luyện: cho hai loại đối tượng vào cùng một danh sách và gọi tuition. **Hướng giải:** phương thức ghi đè được chọn theo đối tượng; không cần chuỗi if kiểm tra mọi tên lớp. Lớp học "có nhiều học viên" không nên kế thừa Student chỉ để dùng lại thuộc tính.

Nguồn học tiếp

https://docs.python.org/3/tutorial/